CHAPTER 6 — SOFTWARE METRICS

Comparative Analysis & Evaluation

Section 1: Sizing & Algorithmic Complexity Comparisons

Q6.C01. Compare Lines of Code (LOC) and Albrecht's Function Points (FP) as software size metrics.

Comprehensive Comparative Analysis:

- **1. Measurement Basis & Underlying Philosophy:**
- **Lines of Code (LOC):** Measures the **physical syntactic volume** of the software artifact (source code text). It reflects how much text developers typed.
- **Function Points (FP):** Measures the **functional business utility** delivered to the user, based on external inputs, outputs, inquiries, internal files, and external interfaces, independent of technology.
- **2. Programming Language Independence:**
- **LOC: Completely language dependent.** A system implemented in Assembly or C requires many times more lines than the same system written in Python or SQL.
- **FP: Completely language independent.** The functional point value remains identical regardless of whether the system is implemented in Java, C++, Python, or COBOL.
- **3. Lifecycle Phase Availability:**
- **LOC: Available only late in the lifecycle.** Exact counts exist only after coding is underway or finished. Early estimation requires error-prone backfiring or proxy guessing.
- **FP: Available early in the lifecycle.** Can be calculated directly from the requirements specification (SRS) and logical data models during analysis, enabling early project bidding and budget planning.
- **4. Productivity Measurement & The Inverse Paradox:**
- **LOC:** Fosters the **inverse productivity paradox** when calculated as LOC/PM. Developers using expressive, concise languages appear "less productive" than developers writing verbose, bloated code.
- **FP:** Provides a functional-size numerator for ratios such as FP/PM. A count of 50 FP is twice the measured functional size of 25 FP under the same counting rules; it does not by itself prove twice the productivity unless effort and scope are comparable.
- **5. Primary Limitations:**
- **LOC:** Disregards algorithmic complexity, code readability, and architecture; discourages code reuse and refactoring.
- **FP:** Involves subjective human judgment in classifying component complexity (Low, Average, High) and rating General System Characteristics; may be less informative when a system's required functional transactions do not capture the algorithmic work of interest; suitability depends on the counting scope.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–149, 250; Companion Slides Ch.4 & Ch.6. Exercises 6.20, 6.25.

Q6.C02. Contrast Halstead's Software Science and McCabe's Cyclomatic Complexity as measures of program complexity.

Comprehensive Comparative Analysis:

- **1. Theoretical Foundation:**
- **Halstead's Software Science:** Grounded in **cognitive psychology and lexical token information theory**. Models code as a static sequence of operator and operand tokens.
- **McCabe's Cyclomatic Complexity:** Grounded in **graph theory and structural topology**. Models code as a directed Control Flow Graph (CFG) of nodes (basic blocks) and edges (transfers of control).
- **2. What is Actually Measured:**
- **Halstead (V, D, E):** Measures **computational vocabulary and informational size**. Difficulty D reflects operand reuse and operator variety.
- **McCabe ($V(G)$):** Measures **decision logic, branching complexity, and path independence**. Directly counts the number of linearly independent execution paths ($E - N + 2P$).
- **3. Treatment of Sequential vs Branching Logic:**
- **Halstead:** Sensitive to sequential statements; fifty consecutive assignment statements produce substantial Volume and Effort.
- **McCabe:** Completely indifferent to sequential statements; fifty consecutive assignment statements possess $V(G) = 1$ (zero branches).
- **4. Testing Relevance:**
- **Halstead:** Provides no guidance on test case generation; effort E estimates mental coding time, not testing coverage.
- **McCabe:** For basis-path testing, $V(G)$ is the number of independent paths in a selected basis; this is not a claim of full condition or decision coverage.
- **5. Empirical Collinearity:**
- **Halstead:** token counts and LOC both describe aspects of program size, but they are not interchangeable; the supplied course material does not establish a universal correlation coefficient.
- **McCabe:** Captures structural risk independent of LOC; a 20-line switch-case statement can have $V(G) = 20$, while a 200-line straight-line function has $V(G) = 1$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–262, 422–425; companion slides 11–23, 132–134.

Q6.C03. Evaluate Halstead's Software Science versus Function Points for project cost and effort estimation.

Comparative Evaluation (Exercise 6.23):

- **1. Applicability at Project Inception:**
- **Function Points: Superior for early cost estimation.** FP can be calculated during initial requirements engineering from user workflows and data entity specifications before a single line of code is written.
- **Halstead's Metrics: Unusable for early project bidding.** Halstead requires counting unique and total operators and operands (η_1, η_2, N_1, N_2), which cannot be determined until code is authored or detailed pseudo-code is finalized.
- **2. Scope of Effort Captured:**
- **Function Points:** Models end-to-end development lifecycle effort, including requirements elicitation, design reviews, coding, testing, user documentation, and project management.
- **Halstead's Effort (E):** A source-code-derived estimate in Halstead's model. It is not a complete measure of total project effort, which also includes activities outside the model's token-based calculation.
- **3. Sensitivity to Technical Architecture:**
- **Function Points:** Evaluates business requirements but may under-represent complex mathematical processing (addressed by Feature Points).
- **Halstead's Metrics:** Reflects low-level algorithmic manipulation but fails to capture macro architectural overhead or database transactions.
- **Strategic Conclusion: Function Points is the industry standard for contract bidding and project cost prediction.** Halstead's Software Science is confined to post-implementation source code analysis and academic compiler studies.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–149, 255–262; Companion Slides Ch.4 & Ch.6. Exercise 6.23.

Q6.C04. Compare C and Pascal implementations of algorithms under Halstead's Software Science metrics.

Comparative Analysis (Exercise 6.9):

- **1. Language Expressiveness and Language Level (λ):**
- **C Language:** Offers rich, terse compound operators (++, --, +=, *=, bitwise operators, pointer arithmetic) and concise control syntax. A C implementation typically features a higher operator vocabulary (η_1) but requires significantly fewer total tokens (N) to express identical logic.
- **Pascal Language:** Designed as a verbose, strongly typed pedagogical language. Lacks compact update operators; requires explicit, verbose syntax (e.g., `x := x + 1;` instead of `x++;`, explicit begin ... end blocks).
- **2. Impact on Vocabulary (η) and Length (N):**
- For identical algorithmic logic, the **Pascal implementation yields a substantially higher total length** N due to verbose block delimiters and assignment statements.
- C exhibits a slightly larger operator vocabulary η_1, but its total length N is significantly smaller.
- **3. Volume (V) and Effort (E):**
- Because Volume $V = N \log_2 \eta$, **the larger length N in Pascal results in greater Program Volume V.**
- However, because C utilizes dense, multifaceted pointer manipulation and side-effect expressions, its Program Difficulty $D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$ is frequently higher per line of code than Pascal's straightforward, transparent syntax.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 260–265; companion slides 23–38. Exercise 6.9.

Section 2: Architectural, Structural & Information Flow Comparisons

Q6.C05. Contrast Traditional Procedural Metrics with Object-Oriented Metrics.

Comprehensive Comparative Analysis:

- **1. Fundamental Unit of Decomposition:**
- **Procedural Metrics (LOC, Halstead, McCabe):** Analyze **standalone subroutines, subprograms, or modules** executing step-by-step sequential algorithms.
- **Object-Oriented Metrics (CK, MOOD, Lorenz–Kidd):** Analyze **classes, object interactions, and inheritance hierarchies** encapsulating state and behavior together.
- **2. Inadequacy of Traditional Metrics in OO Systems:**
- **Small Method Paradox:** Well-designed OO classes consist of numerous small, highly cohesive methods (often 3 to 10 LOC). Applying McCabe's metric yields $V(G) = 1 \text{ or } 2$ for virtually every method, falsely indicating trivial complexity while masking systemic complexity.
- **Complexity Relocation:** In OO architectures, complexity shifts away from internal method algorithmic logic toward **inter-object communication, message-passing protocols, and inheritance depth**. Traditional metrics cannot measure inheritance or dynamic binding.
- **3. Architectural Attributes Measured:**
- **Procedural Metrics:** Control flow branching ($V(G)$), lexical tokens (V, E), and module call fan-in/fan-out.
- **OO Metrics:** Class encapsulation (LCOM, AHF), inheritance reuse and fragile base class risks (DIT, NOC, MIF), and inter-class coupling (CBO, RFC, CF).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–293; companion slides 78–88. Exercise 6.26.

Q6.C06. Contrast Coupling Between Objects (CBO) and Response For a Class (RFC) in the CK Suite.

Comparative Analysis:

- **1. Measurement Focus & Architectural Scope:**
- **Coupling Between Objects (CBO):** Measures **structural dependency coupling**. It counts the number of other classes to which a target class is explicitly linked through method calls, field accesses, parameter passing, or inheritance.
- **Response For a Class (RFC):** Measures **potential execution response complexity**. It counts the total number of distinct methods that can be executed in response to a message sent to an object of the target class.
- **2. Formulation & Inclusions:**
- **CBO:** Counts **classes** (an integer count of external collaborating class entities).
- **RFC:** Counts **methods**. $RFC = |M \cup \bigcup R_m|$, where M is the set of methods internal to the class and R_m is the set of external methods directly invoked by M.
- **3. Engineering Significance:**
- **High CBO:** Indicates rigid structural coupling; makes the class difficult to reuse independently and highly vulnerable to ripple effects when collaborating classes change.
- **High RFC:** Indicates extensive execution branching and message propagation; substantially increases the testing effort required for integration and debugging.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 289–291; companion slides 82–83. Exercise 6.27.

Q6.C07. Compare Object-Oriented Cohesion Measures: Lack of Cohesion in Methods (LCOM) versus Tight Class Cohesion (TCC) and Loose Class Cohesion (LCC).

Comprehensive Comparative Analysis:

- **1. Underlying Approach & Perspective:**
- **LCOM (Chidamber & Kemerer):** An **inverse cohesion metric** measuring the *absence* of cohesion. Evaluates method pairs that share no instance variables (P) versus pairs that share at least one (Q).
- **TCC & LCC (Bieman & Kang): Direct cohesion metrics** normalized between 0.0 and 1.0 (0% to 100%), measuring the relative density of method connections through shared instance variables.
- **2. Treatment of Direct versus Indirect Connections:**
- **LCOM:** Considers only direct sharing of attributes within method pairs.
- **TCC (Tight):** Considers **only direct connections** (two methods access at least one common instance variable).
- **LCC (Loose):** Considers **both direct and indirect connections** (two methods are connected if there exists a chain of intermediate method accesses connecting them).
- **3. Mathematical Bounds & Invariant:**
- **LCOM:** Under the stated variant, lower values indicate less measured lack of cohesion; $LCOM = 0$ is the minimum value of the metric, not a standalone proof of design quality.
- **TCC & LCC:** Real range $[0.0, 1.0]$. Higher values indicate stronger cohesion.
- **Mathematical Invariant:**

$$0.0 \leq TCC \leq LCC \leq 1.0$$

  LCC is always greater than or equal to TCC because the set of indirect connections contains the direct connections.

- **4. Architectural Interpretation:**
- When $TCC = LCC = 1.0$, the class has maximal cohesion.
- When LCC is higher than TCC, indirect links contribute to the connectivity count. This can prompt inspection of the method/attribute graph; it does not by itself establish that the class should be split.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 84–86. Exercise 6.27.

Q6.C08. Evaluate the trade-offs between Reusing Existing Software Components versus Developing New Code from scratch.

Comparative Evaluation (Exercise 6.22):

- **1. Initial Development Cost and Schedule:**
- **Reuse:** Slashes initial construction time, eliminates redundant algorithmic coding, and enables rapid time-to-market.
- **New Development:** Incurs full architectural, design, coding, and debugging costs with significant schedule risk.
- **2. Defect Density and Operational Reliability:**
- **Reuse:** Mature commercial or open-source libraries have undergone thousands of operational execution hours across diverse environments, yielding substantially lower initial defect rates.
- **New Development:** Freshly authored code inevitably contains novel defects and edge-case vulnerabilities requiring extensive testing.
- **3. Architectural Fit and Dependency Risk:**
- **Reuse:** Introduces external architectural dependencies, potential version conflicts ("dependency hell"), unused code bloat, and security vulnerabilities embedded in third-party supply chains.
- **New Development:** Provides perfect alignment with exact business requirements, complete source code control, and zero extraneous dependencies.
- **4. Long-Term Maintenance and Cognitive Overhead:**
- **Reuse:** Maintenance depends on third-party maintainers. If an upstream library is abandoned or undergoes breaking API changes, maintenance costs spike dramatically.
- **New Development:** Maintenance is fully in-house, but the organization bears 100% of the ongoing lifecycle support cost.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 10, 74. Exercise 6.22.

Q6.C09. Evaluate whether Data Structure Metrics measure complexity, quality, or both.

Comparative Evaluation (Exercise 6.21):

- **1. Perspective as a Complexity Measure:**
- **Span (SP) and Live Variables (LV):** Directly quantify the **cognitive complexity and working memory load** imposed on a programmer reading or modifying code. When many variables remain live simultaneously (LV_{max}), the programmer must track multiple state values across wide spans (SP_{avg}), directly reflecting high mental complexity.
- **2. Perspective as a Quality Measure:**
- **Module Weakness ($WM = \overline{LV} \times \gamma$):** Serves as an indirect predictor of **modifiability, readability, and defect-proneness**. Wide spans and long lifespans increase the likelihood of state corruption.
- **Synthesis: Data structure metrics measure both complexity and quality simultaneously.** They measure *internal cognitive complexity* directly (working memory burden) and serve as a *predictor metric for external product quality* (defect vulnerability and maintainability).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 273–280; companion slides 39–68. Exercise 6.21.

Q6.C10. Contrast Defect Density and Defect Removal Efficiency (DRE) as quality indicators.

Comparative Analysis:

- **1. Measurement Scope & Entity:**
- **Defect Density:** A **product quality metric** measuring the normalized concentration of defects within a completed software deliverable ($\text{Defects}/KLOC$ or $\text{Defects}/FP$).
- **Defect Removal Efficiency (DRE):** A **process quality metric** measuring the filtering effectiveness of quality assurance gates ($DRE = E/[E + D]$).
- **2. Role in Process Improvement:**
- **Defect Density:** Identifies which specific modules or subsystems are defect-prone and require refactoring or additional testing.
- **DRE:** Evaluates whether inspection, review, and testing techniques are successfully catching defects before software reaches the customer.
- **3. Management Insight:**
- A low Defect Density in released software could result from two entirely different causes: (a) genuinely high-quality design, or (b) inadequate testing that failed to discover existing bugs.
- Examining **DRE in conjunction with Defect Density** can help distinguish poor quality from inadequate defect discovery. Interpret both against an agreed project baseline; neither alone proves the process is robust.

Source: Pressman, R., 5th ed., printed pp. 98, 126; K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 10.

Summarize the five major trade-offs between LOC and Function Points.
Explain why McCabe's Cyclomatic Complexity is indifferent to sequential code while Halstead's Volume is highly sensitive.
State the mathematical relationship and bounding inequality between TCC and LCC.
Explain the small method paradox and why traditional procedural metrics fail in object-oriented architectures.
Justify why data-structure metrics function simultaneously as complexity and quality indicators.